LAPORAN PRAKTIKUM STRUKTUR DATA

“DOUBLE LINKED LIST”

Disusun Oleh :

Dzhillan Dzhalila

2511531001

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Muhammad Galid A.

FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS
2026

KATA PENGANTAR

Puji syukur penulis panjatkan atas kehadirat Allah SWT Tuhan Yang Maha Esa yang telah memberikan rahmat dan berkatnya, sehingga penulis dapat menyelesaikan laporan praktikum ini yang telah dilaksanakan sebagai pertemuan ke-6 untuk mata kuliah Praktikum Struktur Data. Praktikum ini berfokus pada pembahasan konsep struktur data Stack dengan implementasi Double Linked List.

Melalui praktikum ini, hasil yang diharapkan adalah penulis dapat memahami dan mengimplementasikan penggunaan Double Linked List dalam real Project. Melalui projek projek sederhana yang dilakukan pada praktikum ini membantu mengembangkan pemahaman akan Double Linked List pada program.

Penulis menyadari bahwa, laporan praktikum ini masih memiliki banyak kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan dan pengembangan di masa mendatang. Semoga laporan ini dapat memberikan manfaat, baik sebagai bahan pembelajaran penulis maupun referensi pembaca dalam memahami dasar-dasar pemrograman.

Padang, 2026

Dzhillan Dzhalila

DAFTAR ISI

KATA PENGANTAR i
DAFTAR ISI ii
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan 1
1.3 Manfaat 1
BAB II PEMBAHASAN 2
2.1 Dasar Teori 2
2.2 Program NodeDLL_2511531001 2
2.2.1 Kode Program 2
2.2.2 Penjelasan 3
2.3 Program InsertDLL_2511531001 3
2.3.1 Kode Program 3
2.3.2 Penjelasan 5
2.3.3 Output 8
2.4 Program Penelusuran DLL_2511531001 8
2.4.1 Kode Program 8
2.4.2 Penjelasan 9
2.4.3 Output 11
2.5 Program HapusDLL_2511531001 11
2.5.1 Kode Program 11
2.5.3 Penjelasan 13
2.5.4 Output 15
BAB III KESIMPULAN 16
DAFTAR PUSTAKA 17
LAPORAN TUGAS 18

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman dan pengembangan perangkat lunak, struktur data menjadi salah satu pilar utama dalam mencapai tingkat efisiensi dalam perfoma algoritma dalam pengembangan tersebut. Double Linked List adalah salah satu struktur data yang menawarkan fleksibilitas lebih tinggi dibandingkan dengan Single Linked List biasa. Double Linked List adalah struktur data linear yang terdiri dari Kumpulan node di mana setiap node memiliki tiga komponen utama. Komponen utama double Linked List adalah field untuk menyimpan data, pointer next yang menunjuk ke node berikutnya dan pointer prev yang menunjuk ke node sebelumnya. Kemampuan navigasi dua arah ini menjadikan Double Linked List sangat cocok untuk aplikasi yang membutuhkan akses bolak balik terhadap data, seperti fitur undo-redo dan juga navigasi forward- backward.

1.2 Tujuan

1. Memahami konsep dasar dari struktur data Double Linked List.
2. Mengidentifikasi penggunaan Double Linked List dalam program.
3. Mengimplementasikan metode bawaan dari Double Linked List.

1.3 Manfaat

1. Memberikan pengalaman langsung dalam mengimplementasikan struktur data Double Linked List.
2. Melatih ketepatan dalam memilih struktur data untuk menghasilkan program yang lebih efisien dan terstruktur

BAB II
PEMBAHASAN

2.1 Dasar Teori

Double Linked List adalah linked list yang setiap nodenya memiliki dua pointer dimana strukturnya adalah dimulai dan diakhir dengan nilai null dan di dalamnya ada data node node. Double Linked List memiliki navigasi 2 arah yang diberikan dengan adanya pointer next untuk mengarah ke node selanjutnya dan juga pointer prev untuk mengarah ke node sebelumnya. Kelebihan double linked list adalah bisa bergerak maju dan mundur dan bisa melakukan penghapusan node dengan lebih mudah. Tetapi, double linked list akan membutuhkan memori lebih besar dan memiliki struktur lebih kompleks.

Penggunaan double linked list biasanya digunakan dalam fitur undo ataupun rendo pada aplikasi. Navigasi halaman pada browser yang memungkin kita untuk melihat history halaman browse yang sudah kita buka. Serta sistem playlist lagu yang memungkinkan kita untuk melihat urutan playlist dari awal ataupun dari akhir playlist.

2.2 Program NodeDLL_2511531001

2.2.1 Kode Program

```
package pekan6_2511531001;
public class NodeDLL_2511531001 {
	//mendefinisikan kelas node
	int data_1001; // data
	NodeDLL_2511531001 next_1001; // pointer ke next
mode
	NodeDLL_2511531001 prev_1001; // pointer ke previous
node

	//konstruktor
	public NodeDLL_2511531001 (int data_1001) {
		this.data_1001= data_1001;
		this.next_1001 = null;
		this.prev_1001 = null;
	}

}
```

Kode 2.1 : Kode Program ADT NodeDLL_2511531001

2.2.2 Penjelasan

Program di atas adalah program ADT yang dimulai dari pendeklarasian dari variabel yang akan digunakan untuk program Double Linked List selanjutnya. Variabel yang dideklarasikan adalah data sebagai tempat penyimpanan elemen dari list dan bertipe data integer, next sebagai pointer untuk ke node selanjutnya dan juga prev sebagai pointer untuk node sebelumnya. Setelah itu ada konstruktor untuk membuat ketiga variabel tersebut menjadi valid. Pada awalnya variabel next dan prev akan bernilai null terlebih dahulu, sementara variabel data akan berisi dengan data node yang telah dimasukkan.

2.3 Program InsertDLL_2511531001

2.3.1 Kode Program

```
package pekan6_2511531001;
public class InsertDLL_2511531001 {
	//menambahkan node di awal kiri
	static NodeDLL_2511531001 InsertBegin_1001
(NodeDLL_2511531001 head_1001, int data_1001) {
		//buat node baru
		NodeDLL_2511531001 new_node_1001 = new
NodeDLL_2511531001 (data_1001);
		//jadikan pointer nextnya head
		new_node_1001.next_1001 = head_1001;
		// jadikan pointer prev head ke new_node
		if (head_1001 != null) {
			head_1001.prev_1001 = new_node_1001;
		}
		return new_node_1001;
	}

	//fungsi menambahkan node di akhir
	public static NodeDLL_2511531001 insertEnd_1001
(NodeDLL_2511531001 head_1001, int newData_1001) {
		//buat node baru
		NodeDLL_2511531001 newNode_1001 = new
NodeDLL_2511531001 (newData_1001);
		//jika dll null jadikan head
		if(head_1001 == null) {
			head_1001 = newNode_1001;
		}
		else {
			NodeDLL_2511531001 curr_1001 =
head_1001;
```

```
                    while (curr_1001.next_1001 != null) {
                            curr_1001 = curr_1001.next_1001;
                    }
                    curr_1001.next_1001 = newNode_1001;
                    newNode_1001.prev_1001 = curr_1001;
            }
            return head_1001;
    }

    //fungsi menambahkan node  di posisi tertentu
    public static NodeDLL_2511531001
insertAtPosition_1001 (NodeDLL_2511531001 head_1001, int
pos_1001, int new_data_1001) {
            //buat node baru
            NodeDLL_2511531001 new_node_1001 = new
NodeDLL_2511531001 (new_data_1001);
            if (pos_1001== 1) {
                    new_node_1001.next_1001 = head_1001;
                    if (head_1001 != null) {
                            head_1001.prev_1001 =
new_node_1001;}
                    head_1001 = new_node_1001;
                    return head_1001;
                    }
            NodeDLL_2511531001 curr_1001 = head_1001;
            for ( int i = 1; i <pos_1001 -1 &&
curr_1001 != null; i++) {
                    curr_1001 = curr_1001.next_1001;}
            if(curr_1001 == null) {
                    System.out.println("Posisi tidak ada");
                    return head_1001;
            }

            new_node_1001.prev_1001 = curr_1001;
            new_node_1001.next_1001 =
curr_1001.next_1001;
            curr_1001.next_1001 = new_node_1001;
            if (new_node_1001.next_1001 != null) {
                    new_node_1001.next_1001.prev_1001 =
new_node_1001;
            }
            return head_1001;
            }

    public static void printList_1001
(NodeDLL_2511531001 head_1001) {
            NodeDLL_2511531001 curr_1001 = head_1001;
            while (curr_1001 != null) {
                    System.out.print(curr_1001.data_1001 +
" <-> ");
                    curr_1001 = curr_1001.next_1001;
            }
            System.out.println();
    }

    public static void main (String [] args) {
            //membuat dll 2 <-> 1 <-> 5
```

```
            NodeDLL_2511531001 head_1001 = new
NodeDLL_2511531001(2);
            head_1001.next_1001 = new
NodeDLL_2511531001(3);
            head_1001.next_1001.prev_1001 = head_1001;
            head_1001.next_1001.next_1001 = new
NodeDLL_2511531001 (5);
            head_1001.next_1001.next_1001.prev_1001 =
head_1001.next_1001;
            //cetak DLL awal
            System.out.print("DLL Awal : ");
            printList_1001 (head_1001);
            //tambah 1 di awal
            head_1001 = InsertBegin_1001 (head_1001,1);
            System.out.print("simpul 1 ditambah di awal:
");
            printList_1001(head_1001);
            //tambah 6 di akhir
            System.out.print("simpul 6 ditambah di akhir:
");
            int data_1001 = 6;
            head_1001 = insertEnd_1001 (head_1001,
data_1001);
            printList_1001(head_1001);
            //menambah node 4 di posisi 4
            System.out.print("tambah node 4 di posisi 4:
");
            int data2_1001 = 4;
            int pos_1001 = 4;
            head_1001 = insertAtPosition_1001 (head_1001,
pos_1001, data2_1001);
            printList_1001(head_1001);
            }

}
```

Kode Program 2.2 : Program InsertDLL

2.3.2 Penjelasan

Program ini dimulai dengan mendeklarasikan method InsertBegin dengan variabel output head dan variabel data. Dalam method ini juga dibuat variabel untuk menyimpan node baru yang akan dimasukkan ke dalam list, variabel ini dinamakan new_node. Setelah itu pointer next untuk variabel one_node akan dijadikan sebagai head dalam list. Setelah itu pointer prev dari variabel head jika head kosong akan menjadi new_node. Setelah itu method ini diakhir oleh pengembalian ned_node dengan return. Method ini akan digunakan untuk memasukkan node baru pada awal list.

Selanjutnya adalah pendeklarasian method insertEnd yang mana akan menggunakan variabel head dan newData sebagai varaiebl inputan. Setelah itu kita juga akan membuat variabel untuk menyimpan node baru yang tetap bernama new_node. Dan jika list DLL kosong atau variabel head kosong, maka head akan diisi oleh newNode. Namun jika list DLL sudah ada elemennya maka variabel curr akan menjadi head dari list, tetapi ketika pointer next pada vairabel curr itu tidak kosong maka variabel curr akan menyimpan node pada pointer setelah curr. Setelah itu node yang disimpan pada pointer next variabel curr akan disimpan sebagai newNode. Dan node yang disimpan dalam pointer prev untuk variabel newNode akan disimpan ke dalam variabel curr. Setelah itu method ini akan mengembalikan dengan return nilai variabel head yang tersimpan. Method insertEnd ini berfungsi untuk menambahkan nilai node tersebut ke akhir

Method selanjutnya adalah insertAtPosition yang membutuhkan variabel head, variabel pos dengan tipe data integer untuk menyimpan posisi dari list dan juga variabel new_data dengan tipe data integer. Setelah itu membuat variabel new_node untuk menyimpan node baru dengan hubungan ke kelas NodeDLL. Setelah itu penggunaan percabangan untuk mengetes apabila variabel posisi diisi dengan 1, maka pointer next pada variabel new_node akan dijadikan head, dan apabila head tidak kosong maka pointer prev dari variabel head akan disimpan dalam new_node, dan pada akhirnya nilai yang disimpan pada head akan sama dengan nilai yang disimpan pada variabel new_node dan akan dikembalikan dengan perintah return untuk nilai dari head. Setelah itu variabel curr dibuat yang mana akan menyimpan nilai variabel head dan akan menggunakan perulangan dimana I dimulai dari 1 dan i masih kurang dari jika nilai variabel pos dikurang dengan 1 dan juga nilai pada variabel curr bernilai null, maka variabel curr akan menyimpan nilai pointer next pada curr. Setelah itu ketika variabel curr ternyata

bernilai null, maka program akan menampilkan pesan posisi tidak ada. Method ini juga akan berakhir dengan mengembalikan nilai variabel head dengan perintah return. Setelahnya pointer dari prev dari variabel new_node akan disimpan dalam variabel curr. Pointer next dari variabel new_node akan disimpan dalam pointer next pada variabel curr. Dan pointer next pada variabel curr akan disimpan dalam variabel new_node. Jika variabel new_node tidak kosong maka variabel new_node dengan kedua pointer akan disimpan sebagai new_noode. Method ini akan mengembalikan nilai head dengan perintah return. Fungsi dari method ini adalah untuk memungkinkan program untuk memasukkan nilai node baru pada posisi yang diinginkan.

Dalam program ini juga ada method printList yang membutuhkan variabel head dimana variabel curr akan dideskripsikan sebagai variabel head. Lalu akan dilakukan pengujian dengan perulangan while saat variabel curr tidak kosong maka program akan menampilkan nilai node yang tersimpan dalam curr dan data dan ditambah dengan tanda <-> dimana nilai curr akan disimpan sebagai nilai pada pointer curr. Lalu program akan menampilkan baris kosong..

Setelah itu adalah kelas main dari program, yaitu diawali dengan mendeklarasikan head sebagai variabel DLL. Lalu memasukkan nilai 2,3,5 ke dalam list. Lalu program akan memanggil printList sebagai penampil untuk DLL awal. Setelah itu program memanggil method insertBegin untuk memasukkan nilai 1 kepada variabel head list dan memanggil printList untuk menampilkan list setelah adanya penambahan. Lalu adanya pendeklarasian variabel data dengan nilai 6 dan memanggil method insertEnd untuk memasukkan nilai 6 ke akhir dari list, lalu hasil list ditambahkan. Lalu, adanya pendeklarasian data2 dengan nilai 4 dan pos 4 untuk menambahkan nilai 4 ke dalam posisi node ke 4 dalam list. Untuk penambahan ini, program membutuhkan pemanggilan

method insertAtPosition. Lalu terakhir program akan menampilkan list setelah semua jenis penambahan dengan memanggil method printList.

2.3.3 Output

```
Javadoc  Declaration  Console ×  Progress
<terminated> InsertDLL_2511531001 [Java Application] C:\Users\ASUS\.p2\pool\plugins\org
DLL Awal : 2 <-> 3 <-> 5 <->
simpul 1 ditambah di awal: 1 <-> 2 <-> 3 <-> 5 <->
simpul 6 ditambah di akhir: 1 <-> 2 <-> 3 <-> 5 <-> 6 <->
tambah node 4 di posisi 4: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <->
```

Gambar 2.1 : Output Program

2.4 Program Penelusuran DLL_2511531001

2.4.1 Kode Program

```java
package pekan6_2511531001;

public class PenelusuranDLL_2511531001 {
	static void forwardTraversal_1001
(NodeDLL_2511531001 head_1001) {
		//memulai penulusuran dari head
		NodeDLL_2511531001 curr_1001 = head_1001;
		//lanjutkan sampai akhir
		while (curr_1001 != null) {
			//print data
			System.out.print(curr_1001.data_1001 +
" <-> ");
			//pindah ke node berikutnya
			curr_1001 = curr_1001.next_1001;
		}
		//print spasi
		System.out.println();
	}
	//fungsi penulusuran mundur
	static void backwardTraversal_1001
(NodeDLL_2511531001 tail_1001) {
		//mulai dari akhir
		NodeDLL_2511531001 curr_1001 = tail_1001;
		//lanjut sampai head
		while (curr_1001 != null) {
			//cetak data
			System.out.print(curr_1001.data_1001 +
" <-> ");
			//pindah ke node sebelumnya
			curr_1001 = curr_1001.prev_1001;
		}
```

```
            //cetak spasi
            System.out.println();
        }

        public static void main (String [] args) {
            //cetak DLL
            NodeDLL_2511531001 head_1001 = new
NodeDLL_2511531001 (1);
            NodeDLL_2511531001 second_1001 = new
NodeDLL_2511531001 (2);
            NodeDLL_2511531001 third_1001  = new
NodeDLL_2511531001 (3);

            head_1001.next_1001 = second_1001;
            second_1001.prev_1001 = head_1001;
            second_1001.next_1001 = third_1001;
            third_1001.prev_1001 = second_1001;

            System.out.println("Penelusuran maju :");
            forwardTraversal_1001(head_1001);

            System.out.println("Penelusuran mundur :");
            backwardTraversal_1001(third_1001);

        }

}
```

Kode 2.3 : Program PenelusuranDLL

2.4.2 Penjelasan

Program ini dimulai dengan membuat suatu kelas static yang bernama forwardTraversal yang membutuhkan variabel head. Setelah itu menginisiasikan untuk variabel curr itu akan sama dengan nilai variabel head yang telah dideklarasikan oleh kelas NodeDLL untuk memulai penelusuran dari node head. Lalu melakukan perulangan dengan perintah while dimana selama node curr belum kosong, maka program akan terus untuk menampilkan data node yang tersimpan pada curr.data dengan tambahan tanda <->, lalu setelahnya penelusuran akan berpindah ke node berikutnya. Setelah perulangan selesai maka program akan menampilkan spasi. Seperti namanya method ini akan melakukan penelusuran dari awal atau head list hingga semua node berhasil ditelusuri.

Method kedua pada program ini adalah backwardTraversal dimana membutuhkan variabel tail. Method ini bertugas untuk melakukan penelusuran yang diawali dari akhir atau tail dari list.

Dimulai dari inisiasi curr sebagai nilai yang disimpan di variabel tail untuk memulai penelusuran. Penelusuran akan dilanjutkan sampai akhir node dengan perulangan while selama variabel curr tidak bernilai kosong, program akan menampilkan data yang tersimpan di curr.data dengan tambah <->. Barulah program akan beralih ke node selanjutnya. Setelah perulangan selesai, maka program akan mencetak satu baris dan spasi untuk method ini.

Lalu untuk kelas main pada program ini dimulai dengan memasukkan nilai 1 sebagai nilai head, 2 untuk nilai second, dan juga 3 sebagai nilai third. Lalu membuat pointer next pada variabel head akan disimpan pada variabel second, dan pointer prev variabel second akan disimpan menjadi head dalam list. Setelah itu pointer next pada variabel second akan disimpan ke dalam variabel third. Untuk pointer prev pada variabel third akan disimpan ke dalam variabel second. Lalu program akan method forwardTraversal dengan variabel head untuk menampilkan penelusuran maju dan juga memanggil method backwardTraversal dengan variabel third untuk menampilkan penelusuran mundur.

2.4.3 Output

```
<terminated> PenelusuranDLL_251153100
Penelusuran maju :
1 <-> 2 <-> 3 <->
Penelusuran mundur :
3 <-> 2 <-> 1 <->
```

Gambar 2.2 : output program Penelusuran

2.5 Program HapusDLL_2511531001

2.5.1 Kode Program

```
package pekan6_2511531001;
public class HapusDLL_2511531001 {
	//fungsi menghapus node awal
	public static NodeDLL_2511531001 delHead_1001
(NodeDLL_2511531001 head_1001) {
		if (head_1001 == null) {
			return null;
		}
		NodeDLL_2511531001 temp_1001 = head_1001;
		head_1001 = head_1001.next_1001;
		if (head_1001 != null) {
			head_1001.prev_1001 = null;
		}
		return head_1001;
	}
	//fungsi menghapus di akhir
	public static NodeDLL_2511531001 delLast_1001
(NodeDLL_2511531001 head_1001) {
		if (head_1001 == null) {
			return null;
		}
		if (head_1001.next_1001 == null ) {
			return null;
		}
		NodeDLL_2511531001 curr_1001 = head_1001;
		while (curr_1001.next_1001 != null) {
			curr_1001 = curr_1001.next_1001;
		}
		//update pointer previous node
		if (curr_1001.prev_1001 != null) {
			curr_1001.prev_1001.next_1001 = null;
		}
		return head_1001;
	}
```

```
        //fungsi menghapus node posisi tertentu
        public static NodeDLL_2511531001 delPos_1001
(NodeDLL_2511531001 head_1001, int pos_1001) {
                //jika DLL kosong
                if(head_1001 == null) {
                        return head_1001;
                }
                NodeDLL_2511531001 curr_1001 = head_1001;
                //telusuri sampai ke node yang akan dihapus
                for (int i =1; curr_1001 != null &&
i<pos_1001; i++) {
                        curr_1001 = curr_1001.next_1001;
                }
                //jika posisi tidak ditemukan
                if (curr_1001 == null) {
                        return head_1001;
                }
                //update pointer
                if (curr_1001.prev_1001 != null) {
                        curr_1001.prev_1001.next_1001 =
curr_1001.next_1001;
                }
                if (curr_1001.next_1001 != null) {
                        curr_1001.next_1001.prev_1001 =
curr_1001.prev_1001;
                }
                //jika yang dihapus head
                if (head_1001 == curr_1001) {
                        head_1001 = curr_1001.next_1001;
                }
                return head_1001;
        }
        //fungsi mencetak DLL
        public static void printList_1001(NodeDLL_2511531001
head_1001) {
                NodeDLL_2511531001 curr_1001 = head_1001;
                while (curr_1001 != null) {
                        System.out.print(curr_1001.data_1001 +
" ");
                        curr_1001 = curr_1001.next_1001;
                }
                System.out.println();
        }

        public static void main (String [] args) {
                //buat sebuah DLL
                NodeDLL_2511531001 head_1001 = new
NodeDLL_2511531001 (1);
                head_1001.next_1001 = new
NodeDLL_2511531001(2);
                head_1001.next_1001.prev_1001 = head_1001;
                head_1001.next_1001.next_1001 = new
NodeDLL_2511531001 (3);
                head_1001.next_1001.next_1001.prev_1001 =
head_1001.next_1001;
```

```
            head_1001.next_1001.next_1001.next_1001 = new
NodeDLL_2511531001 (4);

      head_1001.next_1001.next_1001.next_1001.prev_1001 =
head_1001.next_1001.next_1001;

      head_1001.next_1001.next_1001.next_1001.next_1001 =
new NodeDLL_2511531001 (5);

      head_1001.next_1001.next_1001.next_1001.next_1001.pr
ev_1001 = head_1001.next_1001.next_1001.next_1001;

            System.out.print("DLL awal :" );
            printList_1001(head_1001);

            System.out.print("Setelah head dihapus :" );
            head_1001 = delHead_1001(head_1001);
            printList_1001(head_1001);

            System.out.print("Setelah node terakhir
dihapus :" );
            head_1001 = delLast_1001(head_1001);
            printList_1001(head_1001);

            System.out.print("menghapus node ke 2 :" );
            head_1001 = delPos_1001(head_1001,2);
            printList_1001(head_1001);
      }
2.5.2
```

2.5.3 Penjelasan

Program ini dimulai dengan pendeklarasian method delHead yang membutuhkan variabel head . Method ini menggunakan percabangan dimana ketika head bernilai null maka program akan langsung mengembalikan nilai null. Lalu ada pendeklarasian variabel temp akan sama nilainya dengan nilai node yang disimpan pada variabel head. Lalu head akan menyimpan variabel pada pointer prev variabel head dengan nilai null. Lalu setelah percabangan selesai methode ini akan mengembalikan nilai variabel head dengan perintah return. Method ini akan memungkinkan program untuk menghapus node awal atau node head pada list.

Method selanjutnya pada program ini adalah delLast dengan menggunakan variabel head. Method ini dimulai dengan percabangan, jika head list bernilai null, maka program akan

langsung mengembalikan nilai kosong. Jika nilai pada pointer next dari variabel head bernilai null, program juga akan mengembalikan nilai null. Setelah itu method ini akan mendeklrasikan variabel curr akan sama dengan nilai yang disimpan pada variabel head. Lalu adanya perulangan ketika pointer next pada variabel curr tidak null maka variabel curr akan menyimpan nilai pointer next dari variabel curr. Setelah perulangan tersebut, method ini membutuhkan percabangan ketika pointer prev dari variabel curr tidak bernilai null maka nilai pada pointer prev dan next dari variabel curr akan bernilai null. Setelah itu method akan mengembalikan variabel head dengan perintah return. Method ini akan memungkinkan program untuk menghapus node terakhir di dalam list.

Method ketiga dari program ini adalah delPos yang membutuhkan variabel head dan juga variabel pos sebagai variabel yang menyimpan nilai posisi dari list.; Method ini diawali dengan percabangan ketika head bernilai null maka program akan mengembalikan variabel head dengan perintah head. Setelah itu program akan menyimpan variabel curr sebagai variabel head. Lalu akan adanya perulangan dimana I dimulai dari 1 dan variabel curr tidak kosong dan i harus kurang dari nilai variabel I maka program akan menyimpan nilai pointer next pada variabel curr ke dalam variabel curr sendiri. Perulangan ini berfungsi untuk menelusuri node sampai node yang akan dihapus ditemukan. Jika posisi tidak ditemukan maka program akan mengupdate pointer. Jika node yang dihapus adalah node head, maka variabel head akan menyimpan nilai pointer next pada variabel curr. Lalu setelah adanya percabangan maka method akan mengembalikan nilai head dengan perintah return. Method ini berfungsi untuk menghapus node yang ada pada posisi tertentu di dalam list.

Method ke empat adalah printList yang membutuhkan head, dimana nilai curr akan disimpan ke dalam variabel head. Setelah itu ada perulangan while dimana selagi variabel curr tidak bernilai null

maka program akan menampilkan data yang disimpan dalam variabel dan setelahnya nilai variabel pointer next curr akan disimpan ke dalam curr. Lalu dalam program ini akan menampilkan baris kosong.

Lalu, kelas main pada program ini diawali dengan memasukkan nilai 1,2,3,4,5 ke dalam list. Lalu memanggil method printList untuk menampilkan list. Lalu program memanggil method delHead untuk menghapus node head yaitu 1 dan memanggil printList kembali untuk menampilkan list setelah head dihapus. Selanjutnya program akan menghapus node terakhir yaitu 5 dengan memanggil delLast dan memanggil method printList ke untuk menampilkan list. Program juga akan menghapus node ke 2 dari list dengan memanggil method delPos untuk menghapus 3 dan menampilkan list setelah penghapusan dengan pemanggilan method printList.

2.5.4 Output

```
<terminated> HapusDLL_2511531001 [Java Application] C
DLL awal :1 2 3 4 5
Setelah head dihapus :2 3 4 5
Setelah node terakhir dihapus :2 3 4
menghapus node ke 2 :2 4
```

Gambar 2.3 : Output Program HapusDLL

BAB III
KESIMPULAN

Dari praktikum yang telah dilaksanakan, ada beberapa hal yang dapat disimpulkan. Diantaranya adalah sebagai berikut :

1. Untuk memulai pembangunan program dengan double linked list adalah membuat pointer next dan pointer prev, serta pembuatan node dengan nama baru.
2. Setelah itu untuk perlunya membuat variabel untuk merepresentasikan node yang berada pada awal list yang biasanya dideklarasikan dengan head, dan juga node akhir pada list dideklarasikan dengan variabel tail.
3. Untuk double linked list, kita dapat menambahkan dan menghapus suatu node dari node paling awal, paling akhir ataupun node pada posisi tertentu.
4. Penelusuran juga dapat dilakukan, baik itu dimulai dari awal list dan juga akhir list.

DAFTAR PUSTAKA

1] D. Bali, "Memahami doubly linked list dalam struktur data dengan mudah," *Medium*, 30-Mar-2021. [Online]. Tersedia: https://daismabali.medium.com/memahami-doubly-linked-list-dalam-struktu-data-dengan-mudah-e9192a3fbacd . [Diakses: 13 Mei 2026].

[2] Telkom University Jakarta, "Linked list: Single, double, dan circular linked list," *Universitas Telkom Jakarta*, 13-Feb-2026. [Online]. Tersedia: https://jakarta.telkomuniversity.ac.id/linked-list-single-double-dan-circular-linked-list/ . [Diakses: 13 Mei 2026].

LAPORAN TUGAS

A. Program Lagu_2511531001

```
package pekan6_2511531001;


public class Lagu_2511531001 {
            //mendefinisikan kelas node
            String judul_1001; // judul lagu
            String penyanyi_1001; // nama penyanyi
            Lagu_2511531001 next_1001; // pointer ke next mode
            Lagu_2511531001 prev_1001; // pointer ke previous
node

            //konstruktor
            public Lagu_2511531001 (String judul_1001, String
penyanyi_1001) {
                  this.judul_1001= judul_1001;
                  this.penyanyi_1001 = penyanyi_1001;
                  this.next_1001 = null;
                  this.prev_1001 = null;
      }
             public String getJudul_1001() {return judul_1001;}
                public String getPenyanyi_1001() {return
penyanyi_1001;}
                public Lagu_2511531001 getPrev () {return
prev_1001;}
                public Lagu_2511531001 getNext() {return
next_1001;}

                public void setJudul_1001 (String judul_1001)
{this.judul_1001 = judul_1001;}
                public void setPenyanyi_1001 (String
penyanyi_1001) {this.penyanyi_1001 = penyanyi_1001;}
                public void setPrev_1001 (Lagu_2511531001
prev_1001) {this.prev_1001 = prev_1001;}
                public void setNext_1001 (Lagu_2511531001
next_1001) {this.next_1001 = next_1001;}}
```

B. Program Musik_2511531001

```
package pekan6_2511531001;

import java.util.Scanner;

public class Musik_2511531001 {
    static Scanner scanner_1001 = new Scanner(System.in); //
scanner untuk input pengguna

    public static Lagu_2511531001 tambahLagu_1001(Lagu_2511531001
head_1001, String judul_1001, String penyanyi_1001) {
        Lagu_2511531001 laguBaru_1001 = new
Lagu_2511531001(judul_1001, penyanyi_1001); // buat node lagu
baru
```

```
        if (head_1001 == null) {
             head_1001 = laguBaru_1001;
             } // jika kosong, jadi head
        else {
             Lagu_2511531001 curr_1001 = head_1001;
        while (curr_1001.next_1001 != null) {
             curr_1001 = curr_1001.next_1001;
             }
        curr_1001.next_1001 = laguBaru_1001;
        laguBaru_1001.prev_1001 = curr_1001;
        } // cari tail, hubungkan ke node baru
        return head_1001; // kembalikan head yang sudah diupdate
    }

    public static Lagu_2511531001
hapusLaguAwal_1001(Lagu_2511531001 head_1001) {
        if (head_1001 == null) {
             System.out.println("Playlist kosong!");
             return null;
             } // cek jika kosong
        System.out.println("Lagu \"" + head_1001.judul_1001 + "\"
dihapus!"); // tampilkan info hapus
        if (head_1001.next_1001 != null) {
             head_1001 = head_1001.next_1001;
             head_1001.prev_1001 = null;
             } // jika ada node berikutnya, pindah head dan putus
prev
        else {
             head_1001 = null;
             } // jika cuma 1 node, jadi null
        return head_1001; // kembalikan head baru
    }

    public static void tampilMaju_1001(Lagu_2511531001 head_1001)
{
        if (head_1001 == null) {
             System.out.println("Playlist kosong!");
             return;
             } // cek kosong
        System.out.println("\n=== Playlist (Maju) ==="); //
header tampilan
        Lagu_2511531001 curr_1001 = head_1001; int nomor_1001 =
1; // inisialisasi traversal
        while (curr_1001 != null) {
             System.out.println(nomor_1001 + ". " +
curr_1001.judul_1001 + " - " + curr_1001.penyanyi_1001);
             curr_1001 = curr_1001.next_1001;
             nomor_1001++;
             } // loop pakai next_1001
        System.out.println("======================="); // footer
    }

    public static void tampilMundur_1001(Lagu_2511531001
head_1001) {
        if (head_1001 == null) {
             System.out.println("Playlist kosong!");
             return;
```

```
            } // cek kosong
        Lagu_2511531001 curr_1001 = head_1001;
        while (curr_1001.next_1001 != null) {
            curr_1001 = curr_1001.next_1001;
            } // cari tail dulu
        System.out.println("\n=== Playlist (Mundur) ==="); //
header tampilan
        int nomor_1001 = 1;
        while (curr_1001 != null) {
            System.out.println(nomor_1001 + ". " +
curr_1001.judul_1001 + " - " + curr_1001.penyanyi_1001);
            curr_1001 = curr_1001.prev_1001; nomor_1001++;
            } // loop mundur pakai prev_1001 (wajib DLL!)
        System.out.println("========================"); // footer
    }

    public static void cariLagu_1001(Lagu_2511531001 head_1001,
String judul_1001) {
        if (head_1001 == null) {
            System.out.println("Playlist kosong!");
            return;
            } // cek kosong
        Lagu_2511531001 curr_1001 = head_1001;
        boolean ditemukan_1001 = false; // inisialisasi flag
        while (curr_1001 != null) {
            if
(curr_1001.judul_1001.equalsIgnoreCase(judul_1001)) {
                System.out.println("Ditemukan: " +
curr_1001.judul_1001 + " - " + curr_1001.penyanyi_1001);
                ditemukan_1001 = true;
                break; } curr_1001 = curr_1001.next_1001; }
// cari linear case-insensitive
        if (!ditemukan_1001) {
            System.out.println("Lagu \"" + judul_1001 + "\"
tidak ditemukan.");
            } // jika tidak ketemu
    }

    public static void tampilkanMenu_1001() {
        Lagu_2511531001 head_1001 = null; int pilihan_1001; //
inisialisasi playlist kosong dan variabel pilihan
        do {
            System.out.println("\n=== Playlist Musik NIM:
2511531001 ===\n1. Tambah Lagu\n2. Hapus Lagu Pertama\n3. Lihat
Playlist (Maju)\n4. Lihat Playlist (Mundur)\n5. Cari Lagu\n6.
Keluar\nPilihan: "); // tampilkan menu
            pilihan_1001 = scanner_1001.nextInt();
scanner_1001.nextLine(); // baca input dan konsumsi newline
            switch (pilihan_1001) {
                case 1:
                    System.out.print("Judul: ");
                    String judul_1001 = scanner_1001.nextLine();
                    System.out.print("Penyanyi: ");
                    String penyanyi_1001 =
scanner_1001.nextLine();
                    head_1001 = tambahLagu_1001(head_1001,
judul_1001, penyanyi_1001);
```

```
                    System.out.println("Lagu berhasil
ditambahkan!");
                break; // kasus tambah lagu
                case 2:
                    head_1001 = hapusLaguAwal_1001(head_1001);
                break; // kasus hapus lagu pertama
                case 3:
                    tampilMaju_1001(head_1001);
                break; // kasus tampil maju
                case 4:
                    tampilMundur_1001(head_1001);
                break; // kasus tampil mundur (fitur DLL)
                case 5:
                    System.out.print("Cari judul: ");
                    String cari_1001 = scanner_1001.nextLine();
                    cariLagu_1001(head_1001, cari_1001);
                break; // kasus cari lagu
                case 6:
                    System.out.println("Terima kasih!");
                break; // keluar
                default: System.out.println("Pilihan tidak
valid!"); // input salah
            }
        } while (pilihan_1001 != 6); // loop sampai pilih keluar
        scanner_1001.close(); // tutup scanner
    }

    public static void main(String[] args)
{ tampilkanMenu_1001(); } // entry point program
}
```

C. Output

```
=== Playlist Musik NIM: 2511531001 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: Rumah Singgah
Penyanyi: Fabio Asher
Lagu berhasil ditambahkan!
```

```
=== Playlist Musik NIM: 2511531001 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: Lihatlah Lebih Dekat
Penyanyi: Yura Yunita

Lagu berhasil ditambahkan!
```

```
=== Playlist Musik NIM: 2511531001 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: Kita'
Penyanyi: Sheila on 7
Lagu berhasil ditambahkan!
```

```
=== Playlist Musik NIM: 2511531001 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3

=== Playlist (Maju) ===
1. Rumah Singgah - Fabio Asher
2. Lihatlah Lebih Dekat - Yura Yunita
3. Kita' - Sheila on 7
========================
```

```
=== Playlist Musik NIM: 2511531001 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 4

=== Playlist (Mundur) ===
1. Kita' - Sheila on 7
2. Lihatlah Lebih Dekat - Yura Yunita
3. Rumah Singgah - Fabio Asher
========================
```

```
=== Playlist Musik NIM: 2511531001 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 2
Lagu "Rumah Singgah" dihapus!
```

```
=== Playlist Musik NIM: 2511531001 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 5
Cari judul: Lihatlah Lebih Dekat
Ditemukan: Lihatlah Lebih Dekat - Yura Yunita
```

```
=== Playlist Musik NIM: 2511531001 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 6
Terima kasih!
```